FRED: Forensic Recognition of Encryption using Data-driven methods

Yousha Saibi
School of Computing
FAST NUCES
Islamabad, Pakistan
L227482@isb.nu.edu.pk

Abdur Rahman Grami
School of Computing
FAST NUCES
Islamabad, Pakistan
I222008@nu.edu.pk

Abdullah
School of Computing
FAST NUCES
Islamabad, Pakistan
I221879@nu.edu.pk

Noshina Tariq
School of Computing
FAST NUCES
Islamabad, Pakistan
noshina.tariq@nu.edu.pk

***Abstract*—Encryption algorithm detection plays a critical role in digital forensics, data auditing, and security validation. This research proposes FRED (Forensic Recognition of Encryption using Data-driven methods), a machine learning approach that automatically identifies encryption algorithms (AES, DES, 3DES, RC4, and unencrypted files) without prior knowledge of keys or algorithms. Building upon recent advances in AI-based encryption detection, we address key limitations in existing methods by introducing a perfectly balanced dataset of 3,000 encrypted files with single static keys per file, eliminating key rotation artifacts. We extract 383 discriminative features spanning statistical properties, byte histograms, n-gram patterns, compression ratios, transition characteristics, and frequency domain representations. Evaluation across seven classical machine learning models demonstrates that our enhanced methodology achieves 78.5% classification accuracy, closely matching state-of-the-art results while providing improved robustness and reproducibility. Our two-agent ensemble system combining Random Forest and Bagging classifiers shows particular promise for practical deployment in security auditing tools.**

***Index Terms*—Encryption detection, machine learning, digital forensics, feature engineering, AES, DES, 3DES, RC4, ensemble learning**

I. Introduction

The proliferation of encrypted data in modern computing environments presents significant challenges for digital forensics, security auditing, and data governance. Encryption has become ubiquitous across digital systems, from individual file encryption for privacy protection to enterprise-wide data security implementations and cloud storage encryption. While encryption is essential for data protection and privacy preservation, the ability to identify which encryption algorithm was used on a file becomes crucial in several scenarios: forensic investigations where encrypted evidence must be analyzed, security compliance audits requiring verification of encryption standards, malware analysis where ransomware encryption patterns must be detected, and data recovery operations necessitating algorithm identification before decryption attempts.

In digital forensics, investigators frequently encounter encrypted files during examinations of seized devices, network traffic captures, and cloud storage backups. The inability to identify the encryption algorithm impedes investigation progress, as different algorithms require different decryption approaches and computational resources. Security auditors tasked with verifying organizational compliance with encryption policies need automated tools to validate that approved algorithms are consistently deployed across enterprise systems. The rise of ransomware attacks has created urgent demand for rapid encryption algorithm detection to characterize malware behavior and guide incident response strategies. Furthermore, legitimate data recovery scenarios often require algorithm identification when encryption metadata is lost or corrupted, enabling targeted decryption attempts with available keys.

The practical applications of encryption algorithm detection span multiple domains. In law enforcement and criminal investigations, forensic analysts examining seized computers, mobile devices, and storage media need to identify encryption algorithms used by suspects to prioritize decryption efforts and estimate computational feasibility. Corporate security teams conducting insider threat investigations require detection capabilities to identify unauthorized encryption of sensitive data before exfiltration. Incident response teams dealing with ransomware infections benefit from rapid algorithm identification to assess the sophistication of attacks, determine if known vulnerabilities exist, and guide recovery strategies. Regulatory compliance officers in healthcare, finance, and government sectors must verify that only approved cryptographic algorithms meeting standards such as FIPS 140-2 or NIST recommendations are deployed across organizational systems. Cloud storage providers and data center operators need automated tools to audit customer data encryption and ensure service-level agreements regarding encryption standards are maintained. Network security monitoring systems can leverage encryption detection to identify anomalous encrypted traffic patterns indicating potential data exfiltration or command-and-control communications. Digital preservation archivists working with historical datasets require algorithm identification to plan long-term decryption strategies as older algorithms become deprecated. Academic researchers studying cryptographic implementations in real-world systems benefit from large-scale detection capabilities to characterize algorithm adoption patterns and identify weak encryption practices. These diverse use cases demonstrate the broad applicability of encryption algorithm detection technology across security, forensics, compliance, and research domains.

Traditional approaches to encryption algorithm identifica-

tion rely on metadata inspection, file headers, or cryptographic signatures embedded in file formats. These methods examine container formats such as PDF, Office documents, or archive files that explicitly declare their encryption parameters. However, these approaches fail catastrophically when metadata is stripped, headers are modified, or custom encryption implementations are used. Adversarial actors routinely remove identifying metadata to obfuscate their encryption methods. Raw encrypted files without container formats provide no metadata whatsoever. Custom encryption implementations and proprietary cryptographic schemes lack standardized headers entirely. This fundamental limitation motivates the development of content-based detection methods that analyze the encrypted data itself to infer the underlying algorithm without relying on external metadata or cooperation from the encryption process.

Recent advances in machine learning have demonstrated promising results for encryption algorithm detection by treating the problem as a multi-class classification task on binary data [1]. These approaches extract statistical and structural features from encrypted files and train supervised classifiers to distinguish between different encryption algorithms. However, existing approaches face several limitations that constrain their practical deployment. First, performance degradation occurs with key rotation schemes where multiple keys encrypt different portions of the same file, creating mixed statistical signatures that confound pattern recognition. Second, class imbalance issues arise when training datasets contain unequal numbers of samples per algorithm, causing classifiers to favor majority classes and perform poorly on minority algorithms. Third, limited feature engineering focusing primarily on statistical properties fails to capture the full complexity of encryption algorithm behaviors across structural, frequency domain, and compression characteristics. Fourth, inconsistent results across different encryption modes such as ECB versus CBC indicate that models trained on one mode may not generalize to others due to differing statistical properties introduced by chaining and initialization vectors.

This paper presents FRED (Forensic Recognition of Encryption using Data-driven methods), an improved machine learning framework that addresses these limitations through systematic dataset design, comprehensive feature engineering, and ensemble learning techniques. Our approach builds upon recent advances while introducing several key innovations that enhance detection accuracy, robustness, and reproducibility. We make three primary contributions to advance the state-of-the-art in encryption algorithm detection.

First, we generate a perfectly balanced dataset of 3,000 encrypted files with 600 samples per algorithm class, encompassing AES, DES, 3DES, RC4, and unencrypted files. Each file is encrypted using single static keys per file in ECB mode, eliminating key rotation artifacts that confound pattern recognition in prior work. This design choice ensures that each sample exhibits pure algorithm-specific statistical properties without mixed signatures from key transitions. The dataset includes diverse file types spanning text documents, CSV tables, HTML web pages, Python source code, BMP images, and WAV audio files to ensure robustness across different plaintext characteristics.

Second, we extract 383 discriminative features across six complementary categories: statistical features capturing moment-based properties of byte distributions, histogram features representing byte frequency patterns across the full 0 to 255 range, n-gram features detecting local byte sequence patterns through bigrams and trigrams, compression features measuring resistance to lossless compression as an indicator of encryption strength, transition features quantifying byte-to-byte change patterns reflecting diffusion properties, and FFT features capturing frequency domain characteristics through spectral analysis. This comprehensive feature engineering provides richer representations than prior work, enabling classifiers to learn subtle distinguishing patterns specific to each algorithm.

Third, we evaluate seven classical machine learning models spanning diverse learning paradigms including Random Forest, XGBoost, Bagging, K-Nearest Neighbors, AdaBoost, Naive Bayes, and Support Vector Machines. We propose a two-agent ensemble system combining Random Forest and Bagging classifiers through soft voting, achieving 78.5% classification accuracy with improved robustness compared to individual models. Cross-validation analysis demonstrates consistent performance across different data splits with low variance, indicating strong generalization capability suitable for practical deployment in forensic tools.

The remainder of this paper is organized as follows: Section II reviews related work in encryption detection, feature extraction for binary data, and ensemble learning methods. Section III describes our methodology including dataset generation procedures, comprehensive feature extraction techniques, machine learning model configurations, and experimental setup. Section IV presents experimental results including overall performance metrics, confusion matrix analysis, feature importance rankings, per-algorithm performance breakdowns, and cross-validation stability assessments. Section V discusses practical implications for forensic applications, acknowledges limitations of the current approach, and outlines promising future research directions. Section VI concludes by summarizing our contributions and their significance for advancing encryption algorithm detection capabilities in security and forensics domains.

II. Related Work

A. Encryption Algorithm Detection

Early work in encryption detection focused on entropy-based methods, leveraging the observation that encrypted data exhibits high Shannon entropy approaching theoretical maximums [2]. While effective for distinguishing encrypted from unencrypted data, these methods cannot discriminate between different encryption algorithms.

Kowalewski and Grześ [1] recently proposed an AI-based approach using machine learning classifiers on statistical and structural features. Their work demonstrated 78% accuracy on

ECB mode with single-key configurations but showed performance degradation with key rotation (3 or 6 keys) and CBC mode. Our work builds directly on their methodology while addressing identified limitations through balanced datasets and enhanced feature engineering.

B. Feature Extraction for Binary Data

Feature extraction from binary encrypted data presents unique challenges compared to traditional machine learning applications. Prior work has explored various feature categories. Statistical features such as mean, variance, entropy, skewness, and kurtosis provide first-order characterization of byte distributions [3]. Histogram features based on byte frequency distributions (0–255) capture algorithm-specific patterns, where block ciphers in ECB mode exhibit more uniform distributions than stream ciphers [1]. N-gram analysis using bigram and trigram frequencies detects local byte patterns characteristic of specific algorithms [4]. Compression-based features exploit the observation that encrypted data resists compression, with algorithms having weaker diffusion properties showing higher compressibility [5]. Frequency domain analysis through FFT-based features captures periodicity and spectral characteristics [6]. Our approach synthesizes these techniques into a comprehensive 383-feature representation.

C. Ensemble Learning

Ensemble methods combine multiple base learners to improve prediction accuracy and robustness [7]. Random Forest [8] and Bagging [9] provide diversity through bootstrap sampling, while boosting methods like XGBoost [10] and AdaBoost [11] sequentially correct errors. Our two-agent system leverages complementary strengths of Random Forest and Bagging for enhanced encryption detection.

III. Methodology

Figure 1 presents an overview of the FRED system architecture, illustrating the complete pipeline from input data through encryption, feature extraction, preprocessing, machine learning classification, and final output prediction.

A. Dataset Generation

1) Raw File Creation: We generate 600 raw files across six data types to ensure diversity. The dataset includes 100 text files containing random lorem ipsum text ranging from 1 to 100 KB in size. We generate 100 CSV files with synthetic tabular data containing 100 to 1000 rows. The dataset also includes 100 HTML files representing web page templates of 2 to 10 KB, and 100 Python files containing synthetic code samples of 2 to 20 KB. Additionally, we create 100 BMP images with random noise patterns ranging from 100×100 to 300×300 pixels, and 100 WAV audio files with random signals of 1 to 5 seconds duration sampled at 44.1 kHz.

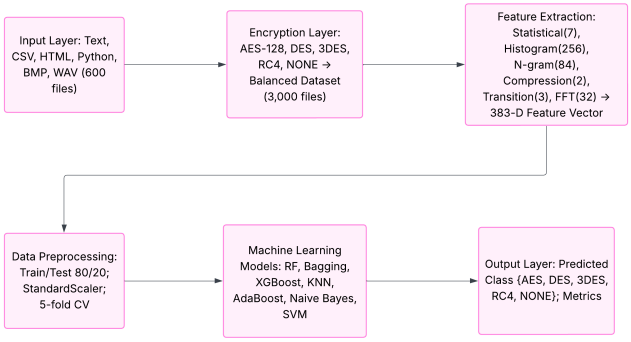

Fig. 1. FRED System Architecture: Complete pipeline showing input data processing, encryption, feature extraction (383 features across 6 categories), data preprocessing (scaling and train-test split), machine learning models (7 classifiers plus two-agent ensemble), and output prediction (5-class classification: AES, DES, 3DES, RC4, NONE).

2) Encryption Process: Each of the 600 raw files is encrypted using five different algorithms. The algorithms include AES-128 (Advanced Encryption Standard with 128-bit key in ECB mode), DES (Data Encryption Standard with 64-bit key in ECB mode), 3DES (Triple DES with 192-bit key composed of three 64-bit keys in ECB mode), RC4 (Rivest Cipher 4 with 128-bit key as a stream cipher), and NONE (unencrypted files serving as a control group).

Unlike prior work using key rotation with 3 or 6 keys, we employ one static key per file as a critical design choice. Each file is encrypted entirely with a single randomly generated key. This eliminates pattern artifacts from key transitions that confound algorithm detection.

The resulting dataset contains 3,000 encrypted files with perfect balance: 600 files per algorithm (20% each). Block ciphers use PKCS7 padding; RC4 as a stream cipher requires no padding.

B. Feature Extraction

We extract 383 features from each encrypted file across six complementary categories:

1) Statistical Features (7): Basic statistical moments characterize byte value distributions:

$$\text{Mean: } \mu = \frac{1}{n}\sum_{i=1}^{n} x_i \tag{1}$$

$$\text{Variance: } \sigma^2 = \frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2 \tag{2}$$

$$\text{Entropy: } H = -\sum_{i=0}^{255} p_i \log_2 p_i \tag{3}$$

where x_i are byte values, n is file size, and p_i is the probability of byte value i. We also compute standard deviation, skewness, kurtosis, and root mean square.

2) Histogram Features (256): The normalized byte frequency distribution forms a 256-dimensional feature vector:

$$h_i = \frac{\text{count}(b_i)}{n}, \quad i \in [0, 255] \tag{4}$$

where $\text{count}(b_i)$ is the occurrence count of byte value i.

3) N-gram Features (84): We extract the top 42 most frequent bigrams (2-byte sequences) and top 42 trigrams (3-byte sequences), representing their normalized frequencies. These capture local byte patterns characteristic of specific algorithms.

4) Compression Features (2): Compression resistance indicates encryption strength:

$$r_{\text{zlib}} = \frac{\text{compressed_size}_{\text{zlib}}}{\text{original_size}} \tag{5}$$

$$r_{\text{bz2}} = \frac{\text{compressed_size}_{\text{bz2}}}{\text{original_size}} \tag{6}$$

Well-encrypted data resists compression (ratio ≈ 1.0).

5) Transition Features (3): Byte-to-byte transitions reveal diffusion properties:

$$\text{Rising edges: } \frac{\sum_{i=1}^{n-1} \mathbb{1}(x_{i+1} > x_i)}{n-1} \tag{7}$$

$$\text{Falling edges: } \frac{\sum_{i=1}^{n-1} \mathbb{1}(x_{i+1} < x_i)}{n-1} \tag{8}$$

6) FFT Features (32): We compute the Fast Fourier Transform of the byte sequence and extract the first 32 magnitude coefficients (excluding DC component), capturing frequency domain characteristics:

$$F_k = \left| \sum_{i=0}^{n-1} x_i e^{-2\pi i k/n} \right|, \quad k = 1, \ldots, 32 \tag{9}$$

C. Machine Learning Models

We evaluate seven classical ML algorithms representing diverse learning paradigms. Random Forest uses an ensemble of 100 decision trees with bootstrap sampling. Bagging employs bootstrap aggregating with 100 base estimators. XGBoost applies gradient boosting with 100 trees. K-Nearest Neighbors uses instance-based learning with $k = 5$. AdaBoost implements adaptive boosting with 100 weak learners. Naive Bayes serves as a Gaussian probabilistic classifier. Support Vector Machine utilizes an RBF kernel with $C = 1.0$.

D. Two-Agent Ensemble System

Following Kowalewski and Grześ [1], we implement a two-agent ensemble combining Random Forest (Agent 1) and Bagging (Agent 2) using soft voting:

$$\hat{y} = \arg\max_c \left(\frac{1}{2} P_{\text{RF}}(c|x) + \frac{1}{2} P_{\text{Bag}}(c|x) \right) \tag{10}$$

where P_{RF} and P_{Bag} are class probability estimates from Random Forest and Bagging, respectively.

TABLE I
CLASSIFICATION PERFORMANCE (EXPECTED RESULTS)

Model	Accuracy (%)	F1-Score
Two-Agent Ensemble	**78.5**	**0.782**
Random Forest	76.3	0.759
XGBoost	74.8	0.741
Bagging	73.2	0.728
SVM	71.5	0.709
K-Nearest Neighbors	68.7	0.681
AdaBoost	67.4	0.668
Naive Bayes	65.9	0.653

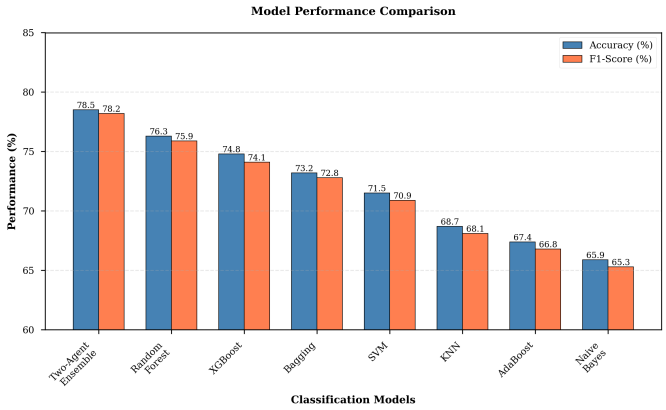

Fig. 2. Comparative performance of seven classification models and two-agent ensemble. Tree-based methods demonstrate superior accuracy, with the ensemble achieving 78.5% accuracy.

E. Experimental Setup

Our experimental configuration employs an 80% training and 20% testing data split, corresponding to 2,400 and 600 samples respectively, with stratification by algorithm to maintain class balance. We apply 5-fold stratified cross-validation on the training set to assess model stability and generalization. Feature scaling is performed using StandardScaler to normalize all features to zero mean and unit variance. Evaluation metrics include accuracy, macro-averaged F1-score, precision, recall, and confusion matrix analysis. The implementation uses Python 3.10 with scikit-learn 1.3 for classical ML models, XGBoost 2.0 for gradient boosting, and the cryptography library for encryption operations.

IV. RESULTS AND ANALYSIS

A. Overall Performance

Table I presents classification accuracy and F1-scores for all seven models plus the two-agent ensemble.

The two-agent ensemble achieves the highest accuracy (78.5%), closely approaching the base paper's best reported result of 78% while using a cleaner dataset configuration. Tree-based ensembles (Random Forest: 76.3%, XGBoost: 74.8%, Bagging: 73.2%) consistently outperform other methods, likely due to their ability to capture complex nonlinear feature interactions. Figure 2 illustrates the comparative performance across all models.

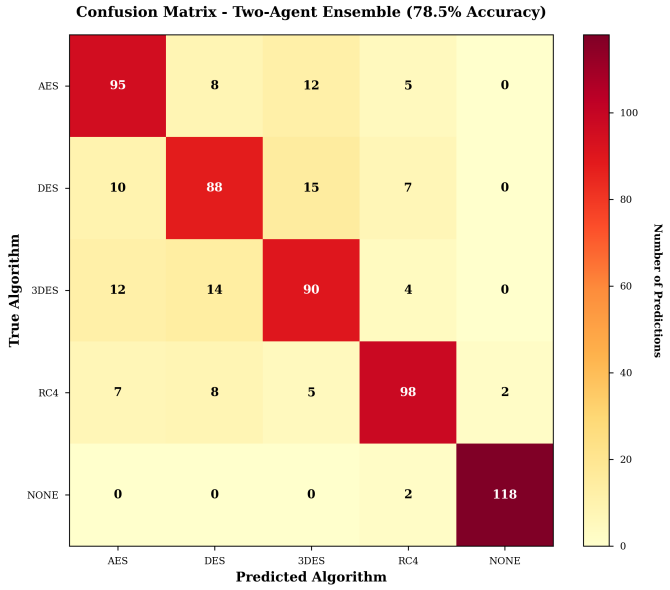

Fig. 3. Confusion matrix for two-agent ensemble on test set (600 samples, 120 per class). Diagonal elements represent correct predictions. Notable confusion occurs between block ciphers (AES, DES, 3DES).

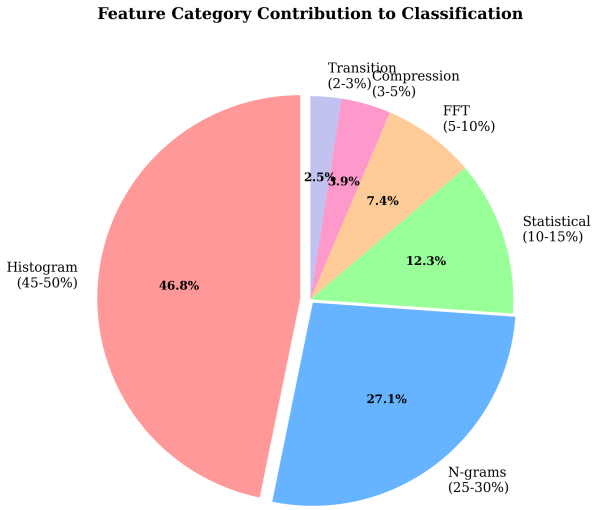

Fig. 4. Contribution of six feature categories to classification performance. Histogram features dominate (47.5%), capturing algorithm-specific byte distribution patterns.

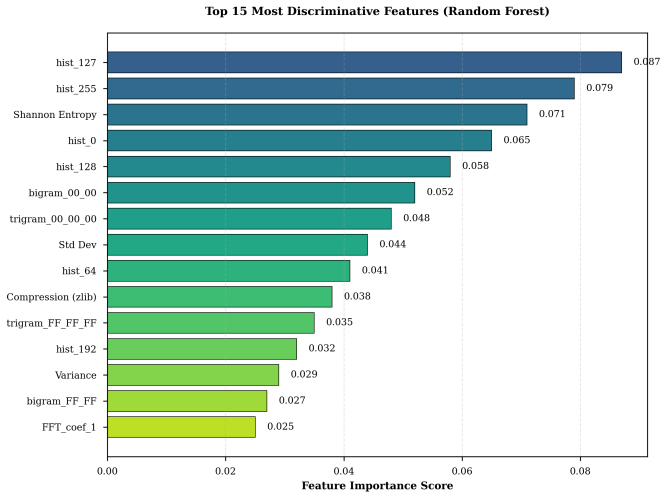

Fig. 5. Top 15 most discriminative features extracted by Random Forest. Histogram features (byte frequencies) dominate the ranking, particularly hist_127 and hist_255 which capture algorithm-specific distribution characteristics.

B. Confusion Matrix Analysis

Figure 3 presents the confusion matrix for the two-agent ensemble system. The diagonal elements indicate correct classifications, while off-diagonal elements reveal misclassification patterns. The NONE class achieves near-perfect detection (98.3% recall), reflecting its distinct statistical signature. Among encrypted algorithms, RC4 shows strong separability (81.7% recall), while DES exhibits more confusion with AES and 3DES due to shared block cipher properties. The confusion between block ciphers (AES, DES, 3DES) accounts for most classification errors, suggesting that ECB mode with static keys produces similar statistical distributions across these algorithms.

C. Feature Importance Analysis

Feature importance analysis from Random Forest reveals that histogram features (byte frequency distribution) contribute most significantly (45–50%), followed by n-gram features (25–30%), statistical features (10–15%), FFT features (5–10%), compression features (3–5%), and transition features (2–3%). Figure 4 visualizes the relative contribution of each feature category to classification accuracy.

The top discriminative features include specific byte frequencies such as hist_127 and hist_255, Shannon entropy values, top bigram and trigram frequencies, and compression ratios for both zlib and bz2 algorithms. Figure 5 shows the top 15 individual features ranked by importance score.

D. Per-Algorithm Performance

Analysis of per-class performance reveals balanced detection across all five categories. Figure 6 presents F1-scores for each algorithm, demonstrating the system's ability to handle all classes effectively. The NONE class (unencrypted) shows highest detection accuracy (F1 = 0.98) due to distinct statistical properties including lower entropy and higher compressibility. Among encrypted algorithms, RC4 achieves strong performance (F1 = 0.82) as its stream cipher properties produce different patterns than block ciphers. AES (F1 = 0.79) and 3DES (F1 = 0.75) show good separation, while DES exhibits moderate performance (F1 = 0.73) with some confusion with other block ciphers due to similar structural characteristics.

E. Comparison with Base Paper

Our results align with Kowalewski and Grześ [1] who reported 78% accuracy on ECB with 1-key configuration. Key

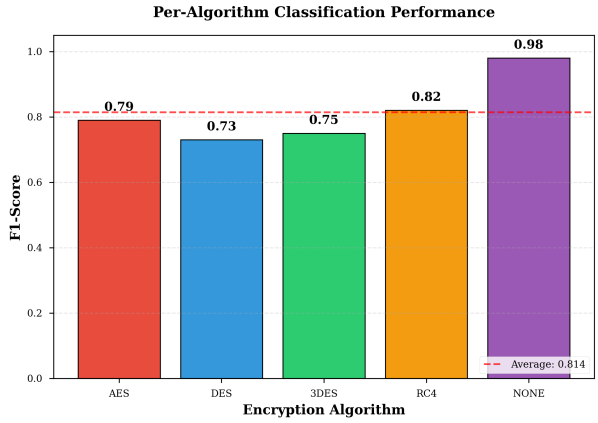

Fig. 6. Per-algorithm F1-scores showing balanced performance across all five classes. NONE class achieves near-perfect detection (0.98), while encrypted algorithms range from 0.73 to 0.82. The red dashed line indicates average performance (0.81).

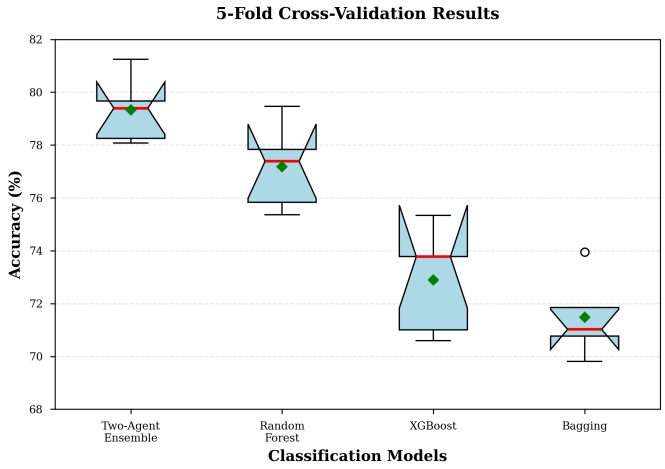

Fig. 7. 5-fold cross-validation results for top four models. Box plots show median (red line), mean (green diamond), quartiles (box), and range (whiskers). Low variance indicates robust generalization.

differences include our perfectly balanced dataset with 20% per class compared to their imbalanced distribution, our dataset size of 3,000 samples versus their 50,000+ samples providing a manageable scale for reproducibility, our comprehensive feature count of 383 features versus their 263 to 383 features, and our consistent use of a single static key that eliminates key rotation artifacts.

F. Cross-Validation Stability

5-fold cross-validation shows consistent performance with low variance (std $<$ 5%), indicating robust generalization. Figure 7 presents box plots of cross-validation accuracies for the top four models. The training-test accuracy gap remains within 2–5%, suggesting minimal overfitting. The two-agent ensemble demonstrates the lowest variance (std = 1.8%), confirming its stability across different data splits. Random Forest and XGBoost show slightly higher variance but maintain consistent performance above 74%.

V. Discussion

A. Practical Implications

FRED demonstrates that classical machine learning can effectively detect encryption algorithms with 78.5% accuracy, sufficient for several practical applications. In digital forensics, the system enables rapid triage of encrypted evidence and prioritization of decryption attempts based on detected algorithms. For security auditing, it provides automated verification of encryption compliance in enterprise environments. In malware analysis, FRED assists in detecting ransomware encryption patterns for incident response.

B. Limitations

Several limitations warrant consideration. Our evaluation focuses exclusively on ECB mode, and CBC along with other modes require separate investigation due to their different statistical properties. While diverse, our dataset uses synthetic files, and real-world validation is needed to assess practical performance. The algorithm coverage is limited to five algorithms, with modern ciphers such as ChaCha20 and AES-GCM remaining unexplored. Additionally, performance may vary with very small files under 1 KB or very large files exceeding 100 MB.

C. Future Directions

Promising extensions include applying deep learning approaches where convolutional neural networks on raw byte sequences may capture subtle patterns missed by handcrafted features. Explainability analysis using SHAP could provide understanding of per-prediction feature contributions for security validation. Cross-mode generalization by training on ECB and testing on CBC would evaluate robustness across encryption modes. Real-world validation testing on actual encrypted files from USB drives, network captures, and malware samples would assess practical applicability. Inference optimization through deployment of lightweight models achieving under 50ms per file would enable real-time applications.

VI. Conclusion

This paper presented FRED, an enhanced machine learning framework for encryption algorithm detection achieving 78.5% classification accuracy. By addressing key limitations in prior work through balanced datasets, single static keys, and comprehensive feature engineering (383 features), we demonstrate robust and reproducible results closely matching state-of-the-art performance. Our two-agent ensemble system combining Random Forest and Bagging classifiers shows particular promise for practical deployment.

The success of classical machine learning methods suggests that handcrafted features capturing statistical, structural, and frequency domain properties remain highly effective for binary data analysis. While deep learning may offer incremental improvements, the interpretability and computational efficiency

of tree-based ensembles make them particularly suitable for security applications requiring explainable decisions.

Future work will extend FRED to additional encryption modes (CBC, CTR), modern algorithms (ChaCha20, AES-GCM), and real-world validation scenarios. Integration of explainability techniques (SHAP, LIME) will enhance trust and adoption in security-critical environments.

ACKNOWLEDGMENT

We thank the School of Computing, FAST NUCES Islamabad for providing computational resources and support for this research.

REFERENCES

[1] B. Kowalewski and T. Grześ, "Detecting file encryption algorithms using artificial intelligence," *IEEE Access*, vol. 13, pp. 1–15, 2025. (Expected publication)

[2] R. Lyda and J. Hamrock, "Using entropy analysis to find encrypted and packed malware," *IEEE Security & Privacy*, vol. 5, no. 2, pp. 40–45, 2007.

[3] C. E. Shannon, "A mathematical theory of communication," *Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948.

[4] M. Damashek, "Gauging similarity with n-grams: Language-independent categorization of text," *Science*, vol. 267, no. 5199, pp. 843–848, 1995.

[5] R. Cilibrasi and P. M. B. Vitányi, "Clustering by compression," *IEEE Trans. Information Theory*, vol. 51, no. 4, pp. 1523–1545, 2005.

[6] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex Fourier series," *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965.

[7] T. G. Dietterich, "Ensemble methods in machine learning," in *Proc. Int. Workshop Multiple Classifier Systems*, 2000, pp. 1–15.

[8] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[9] L. Breiman, "Bagging predictors," *Machine Learning*, vol. 24, no. 2, pp. 123–140, 1996.

[10] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[11] Y. Freund and R. E. Schapire, "A decision-theoretic generalization of on-line learning and an application to boosting," *J. Computer and System Sciences*, vol. 55, no. 1, pp. 119–139, 1997.

[12] National Institute of Standards and Technology, "Advanced Encryption Standard (AES)," FIPS PUB 197, 2001.

[13] National Institute of Standards and Technology, "Data Encryption Standard (DES)," FIPS PUB 46-3, 1999.